

```

1 import torch
2 import torch.nn.functional as nnF
3 import numpy as np
4
5 def burgers_pde_residual(x, t, u):
6     # x: (B, Nx, Nt)
7     # t: (B, Nx, Nt)
8     # u: (B, Nx, Nt)
9
10    #print(x.shape,t.shape)
11    nu = 0.01
12    dt = 1/(t.size()[2]-1)
13    dx = 1/(x.size()[1]-1)
14    # TODO
15    dudx = torch.zeros_like(u)
16    dudt = torch.zeros_like(u)
17    du2dx2 = torch.zeros_like(u)
18    zero_m = torch.zeros_like(u)
19
20    #print(dudx.shape)
21    '''
22    for k in range(0,x.size()[0]-1):
23        for i in range(1,t.size()[2]-1):
24            for j in range(1,x.size()[1]-1):
25
26                dudx[k][j][i] = (u[k, j + 1, i] - u[k
27, j - 1, i]) / (2*dx)
28                du2dx2[k][j][i] = (u[k, j + 1, i] + u
29[k, j - 1, i] - 2 * u[k, j, i]) / dx ** 2
30                dudt[k][j][i] = (u[k, j, i + 1] - u[k
31, j, i - 1]) / (2*dt)
32                #Edges using forward euler
33                dudx[k][0][i] = (u[k, 1, i] - u[k, 0
34, i]) / dx
35                dudx[k][-1][i] = (u[k, -1, i] - u[k
36, -2, i]) / dx
37                dudt[k][j][0] = (u[k, j, 1] - u[k, j
38, 0]) / dt

```

```

33         dudt[k][j][-1] = (u[k, j, - 1] - u[k
    , j, -2]) / dt
34         ##
35         dudx = (u[:, 2:u.size()[1]-1, :] - u[:, 0:u.size
    ()[1]-3, :]) / (2*dx)
36         du2dx2 = (u[:, 2:u.size()[1]-1, :] + u[:, 0:u.
    size()[1]-3, :] - 2 * u[:, 1:u.size()[1]-2, :]) / dx
    ** 2
37         dudt = (u[:, :, 2:u.size()[1]-1] - u[:, :, 0:u.size
    ()[1]-3]) / (2 * dt)'''
38
39         dudx[:, 1:x.shape[1]-2, :] = (u[:, 2:u.size()[1] -
    1, :] - u[:, 0:u.size()[1] - 3, :]) / (2 * dx)
40         du2dx2[:, 1:x.shape[1]-2, :] = (u[:, 2:u.size()[1]
    ] - 1, :] + u[:, 0:u.size()[1] - 3, :] - 2 * u[:, 1:u
    .size()[1] - 2, :]) / dx ** 2
41         dudt[:, :, 1:t.shape[2]-2] = (u[:, :, 2:u.size()[
    1] - 1] - u[:, :, 0:u.size()[1] - 3]) / (2 * dt)
42
43         dudx[:, 0, :] = (u[:, 1, :] - u[:, 0, :]) / (dx)
44         dudx[:, -1, :] = (u[:, -2, :] - u[:, -1, :]) / (
    dx)
45
46         dudt[:, :, 0] = (u[:, :, 1] - u[:, :, 0]) / ( dt)
47         dudt[:, :, -1] = (u[:, :, -2] - u[:, :, -1]) / (
    dt)
48
49         #du2dx2[:, 0, :] = (dudx[:, 1, :] - dudx[:, 0, :])/dx
50         #du2dx2[:, -1, :] = (dudx[:, -2, :] - dudx[:, -1
    , :]) / dx
51
52         #print(dudt.size(), dudx.size(), du2dx2.size())
53
54
55         resid = dudt +0.5* dudx**2 - nu * du2dx2
56         loss = torch.nn.MSELoss()
57         loss = loss(resid, zero_m)
58

```

```
59     bc_loss = torch.nn.MSELoss()
60     bc_loss = bc_loss(u[:,0,:],u[:, -1,:])
61
62     bc_loss2 = torch.nn.MSELoss()
63     bc_loss2 = bc_loss2(dudx[:,0,:],dudx[:, -1,:])
64
65     return loss+bc_loss+bc_loss2
66
67
68 def burgers_data_loss(predicted, target):
69     # Relative L2 Loss
70     # Predicted: (B, Nx, Nt)
71     # Target: (B, Nx, Nt)
72     # TODO
73     num = torch.norm(predicted - target, p =2)
74     den = torch.norm(target,p=2)
75     return num/den
76
77
```

```

1 import torch
2 from torch.utils.data import Dataset
3 import numpy as np
4 from utils import MatReader
5
6
7 class BurgersDataset(Dataset):
8     def __init__(self, data_path, train=True):
9         super().__init__()
10        self.train = train
11        dataloader = MatReader(data_path, to_cuda=
torch.cuda.is_available())
12        self.device = torch.device(
13            'cuda' if torch.cuda.is_available() else
'cpu')
14        self.ic_vals = dataloader.read_field("input"
) # Initial conditions
15        self.solutions = dataloader.read_field("
output") # Solution
16        self.tspan = dataloader.read_field("tspan")
17
18        nx = self.ic_vals.shape[1]
19        nt = self.tspan.shape[1]
20
21        # Discretized Grid (Nx, Nt, 2)
22        self.grid = torch.from_numpy(
23            np.mgrid[0: 1: 1 / nx, 0: 1: 1 / nt]).
permute(1, 2, 0).to(self.device)
24
25        # Quirk of data: Need to transpose x, t
26        self.solutions = self.solutions.permute(0, 2
, 1)
27
28    def __len__(self):
29        # TODO
30        return self.ic_vals.shape[0] # Number of
samples
31

```

```
32     def __getitem__(self, idx):
33         # TODO
34         ic = self.ic_vals[idx, :].to(self.device)
35         sol = self.solutions[idx, :, :].to(self.
device)
36         nx, nt, _ = self.grid.shape
37         ic_broadcast = ic.unsqueeze(1).expand(nx, nt
).unsqueeze(-1)
38         model_input = torch.cat([self.grid,
ic_broadcast], dim = -1)
39
40         return model_input, sol
41
42
```

```
1 import torch
2 import torch.nn as nn
3 import torch.nn.functional as F
4
5
6 class ConvNet2D(nn.Module):
7
8     # You may include additional arguments if you
wish.
9     def __init__(self):
10         # TODO
11         super().__init__()
12         self.conv1 = nn.Conv2d(in_channels=3,
13 out_channels=64, kernel_size=3, stride=1, padding=1)
14         self.Tanh1 = nn.Tanh()
15
16         self.conv2 = nn.Conv2d(64, 64, 3, 1, 1)
17         self.Tanh2 = nn.Tanh()
18
19         self.conv3 = nn.Conv2d(64, 64, 3, 1, 1)
20         self.Tanh3 = nn.Tanh()
21
22         self.conv4 = nn.Conv2d(64, 128, 1)
23         self.Tanh4 = nn.Tanh()
24         self.fc2 = nn.Conv2d(128, 1, kernel_size=1) #
Check during OH
25
26     def forward(self, x):
27         # TODO
28         x = x.permute(0, 3, 1, 2)
29         x = torch.tanh(self.conv1(x))
30         x = self.Tanh2(self.conv2(x))
31         x = self.Tanh3(self.conv3(x))
32         x = self.Tanh4(self.conv4(x))
33         x = self.fc2(x)
34         x = x.squeeze(1)
35         return x
```

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 import torch
4 import torch.nn.functional as nnF
5 import torch.optim as optim
6
7 from train import *
8 from pde import burgers_data_loss
9 device = torch.device('cuda' if torch.cuda.
    is_available() else 'cpu')
10 batch_size = 3
11 burgers_validation = BurgersDataset(
12     'data/Burgers_test_50_visc_0.01.mat', train=
    False)
13 training_losses = np.load("training-losses.npy")
14 #model = torch.load('model.pth')
15 model = torch.load('model.pth',map_location=torch.
    device('cpu'))
16 model.eval()
17 validation_loader = DataLoader(burgers_validation,
    batch_size=batch_size, shuffle=False)
18 #print(validation_loader)
19 val_loss = 0.0
20 num_val_batches = 0
21
22 with torch.no_grad():
23     for x_val, target_val in validation_loader:
24         x_val, target_val = x_val.to(device),
            target_val.to(device)
25         x_val = x_val.float()
26         val_pred = model(x_val)
27         print(x_val.shape)
28         loss_val = burgers_data_loss(val_pred,
            target_val)
29         val_loss += loss_val.item()
30         num_val_batches += 1
31         break
32 print(val_pred.shape, target_val.shape)
```

```
33 print(num_val_batches)
34
35 fig, (ax1, ax2) = plt.subplots(1,2)
36 countour1 = ax1.contourf(val_pred[0,:,:])
37 countour1_t = ax2.contourf(target_val[0,:,:])
38 ax1.set_title('Predicted Solution')
39 ax2.set_title('Target Solution')
40 plt.tight_layout()
41 plt.show(block=False) # Use non-blocking show
42
43 fig2, (ax1, ax2) = plt.subplots(1,2)
44 countour2 = ax1.contourf(val_pred[1,:,:])
45 countour2_t = ax2.contourf(target_val[1,:,:])
46 ax1.set_title('Predicted Solution')
47 ax2.set_title('Target Solution')
48 plt.tight_layout()
49 plt.show(block=False)
50
51 fig3, (ax1, ax2) = plt.subplots(1,2)
52 countour3 = ax1.contourf(val_pred[2,:,:])
53 countour3_t = ax2.contourf(target_val[2,:,:])
54 ax1.set_title('Predicted Solution')
55 ax2.set_title('Target Solution')
56 plt.tight_layout()
57 plt.show(block=False)
58
59 PDE_resid_loss = np.load(f'pde_resid_loss.npy')
60 plt.figure()
61 print(PDE_resid_loss,np.linspace(0,len(PDE_resid_loss
  )))
62
63 plt.plot(np.linspace(0,len(PDE_resid_loss),40),
  PDE_resid_loss)
64 plt.show()
```



```
1 import torch
2 import torch.nn.functional as nnF
3 import torch.optim as optim
4 from tqdm import tqdm
5 import matplotlib.pyplot as plt
6 import numpy as np
7
8 from data import BurgersDataset
9 from model import ConvNet2D
10 from pde import burgers_pde_residual,
    burgers_data_loss
11 from torch.utils.data import DataLoader
12
13 device = torch.device('cuda' if torch.cuda.
    is_available() else 'cpu')
14
15
16 def train():
17     burgers_train = BurgersDataset(
18         'data/Burgers_train_1000_visc_0.01.mat',
19         train=True)
20     burgers_validation = BurgersDataset(
21         'data/Burgers_test_50_visc_0.01.mat', train=
22         False)
23
24     # Hyperparameters
25     lr = 5e-3
26     batch_size = 16
27     epochs = 40
28
29     # Setup optimizer, model, data loader etc.
30     # TODO
31     train_loader = DataLoader(burgers_train,
32                               batch_size=batch_size, shuffle=True)
33     validation_loader = DataLoader(burgers_validation
34                                   , batch_size=batch_size, shuffle=False)
35
36     alpha = 0
```

```

33     model = ConvNet2D().to(device)
34     optimizer = optim.Adam(model.parameters(), lr=lr)
35     training_losses = []
36     pde_resid_loss = []
37     #loss_pde_only = burgers_pde_residual(inputs
    [::,::,0], inputs[:,::,1], target_val)
38     # Training Loop
39     for epoch in range(epochs):
40         model.train()
41         running_loss = 0.0
42         epoch_loss = 0.0
43         running_loss_pde = 0
44         for batch in train_loader:
45             inputs, targets = batch
46             inputs = inputs.float()
47             loss_pde_only = burgers_pde_residual(
    inputs[:, :, :, 0], inputs[:, :, :, 1], targets)
48             #print(inputs.shape)
49             print(loss_pde_only)
50             optimizer.zero_grad()
51             predicted = model(inputs)
52
53             #print(targets.shape)
54             loss1 = burgers_data_loss(predicted,
    targets)
55
56             loss2 = burgers_pde_residual(inputs
    [::,::,0], inputs[:,::,1], predicted)
57             loss = loss1 + alpha * loss2
58             loss.backward()
59             optimizer.step()
60             running_loss_pde += loss_pde_only
61             running_loss += loss.item()
62             epoch_loss += loss.item() * inputs.size(0
    )
63         epoch_loss /= len(train_loader.dataset)
64         training_losses.append(epoch_loss)
65         pde_resid_loss.append(running_loss_pde)

```

```
66
67     print(f"Epoch {epoch + 1}, Loss: {epoch_loss
        :.4f}")
68     np.save(f'training-both_losses.npy', np.array(
        training_losses))
69     np.save(f'pde_resid_loss.npy', np.array(
        pde_resid_loss))
70     torch.save(model, 'model_both_losses.pth')
71     # Validation Loop
72     # TODO
73     model.eval()
74     val_loss = 0.0
75     num_val_batches = 0
76     with torch.no_grad():
77         for x_val, target_val in validation_loader:
78             x_val, target_val = x_val.to(device),
            target_val.to(device)
79             x_val = x_val.float()
80             val_pred = model(x_val)
81             loss_val = burgers_data_loss(val_pred,
            target_val)
82             val_loss += loss_val.item()
83             num_val_batches += 1
84
85     print(f"Validation Loss: {val_loss /
        num_val_batches:.4f}")
86     np.save(f'training-rmses_both_losses.npy', np.
        array(val_loss / num_val_batches))
87     '''
88     plt.figure()
89     plt.plot(range(1, epochs), training_losses,
        label='Data Loss')
90     plt.title('Data Loss vs. Epoch')
91     plt.xlabel('Epochs')
92     plt.ylabel('Data Loss')
93     plt.legend()
94
95     plt.figure()
```

```
96     plt.contourf()
97     plt.plot(x_val[1,:,:], target_val[1,:,:], label
    = 'Data Loss')
98     plt.title('Data Loss vs. Epoch')
99     plt.xlabel('Epochs')
100    plt.ylabel('Data Loss')
101    plt.legend()
102    '''
103 if __name__ == '__main__':
104     torch.manual_seed(0)
105     train()
106
```

```
1 import torch
2 import scipy.io
3 import h5py
4 import numpy as np
5
6 class MatReader(object):
7     def __init__(self, file_path, to_torch=True,
8         to_cuda=False, to_float=True):
9         super(MatReader, self).__init__()
10
11         self.to_torch = to_torch
12         self.to_cuda = to_cuda
13         self.to_float = to_float
14
15         self.file_path = file_path
16
17         self.data = None
18         self.old_mat = None
19         self._load_file()
20
21     def _load_file(self):
22         try:
23             self.data = scipy.io.loadmat(self.
24 file_path)
25             self.old_mat = True
26         except:
27             self.data = h5py.File(self.file_path)
28             self.old_mat = False
29
30     def load_file(self, file_path):
31         self.file_path = file_path
32         self._load_file()
33
34     def read_field(self, field):
35         x = self.data[field]
36
37         if not self.old_mat:
38             x = x[()]
```

```
37         x = np.transpose(x, axes=range(len(x.  
38         shape) - 1, -1, -1))  
39         if self.to_float:  
40             x = x.astype(np.float32)  
41  
42         if self.to_torch:  
43             x = torch.from_numpy(x)  
44  
45             if self.to_cuda:  
46                 x = x.cuda()  
47  
48         return x  
49  
50     def set_cuda(self, to_cuda):  
51         self.to_cuda = to_cuda  
52  
53     def set_torch(self, to_torch):  
54         self.to_torch = to_torch  
55  
56     def set_float(self, to_float):  
57         self.to_float = to_float  
58
```

```

1 import torch
2 import torch.nn.functional as nnF
3 import numpy as np
4
5 def burgers_pde_residual(x, t, u):
6     # x: (B, Nx, Nt)
7     # t: (B, Nx, Nt)
8     # u: (B, Nx, Nt)
9
10    #print(x.shape,t.shape)
11    nu = 0.01
12    dt = 1/(t.size()[2]-1)
13    dx = 1/(x.size()[1]-1)
14    # TODO
15    dudx = torch.zeros_like(u)
16    dudt = torch.zeros_like(u)
17    du2dx2 = torch.zeros_like(u)
18    zero_m = torch.zeros_like(u)
19
20    #print(dudx.shape)
21    '''
22    for k in range(0,x.size()[0]-1):
23        for i in range(1,t.size()[2]-1):
24            for j in range(1,x.size()[1]-1):
25
26                dudx[k][j][i] = (u[k, j + 1, i] - u[k
27, j - 1, i]) / (2*dx)
28                du2dx2[k][j][i] = (u[k, j + 1, i] + u
29[k, j - 1, i] - 2 * u[k, j, i]) / dx ** 2
30                dudt[k][j][i] = (u[k, j, i + 1] - u[k
31, j, i - 1]) / (2*dt)
32                #Edges using forward euler
33                dudx[k][0][i] = (u[k, 1, i] - u[k, 0
34, i]) / dx
35                dudx[k][-1][i] = (u[k, -1, i] - u[k
36, -2, i]) / dx
37                dudt[k][j][0] = (u[k, j, 1] - u[k, j
38, 0]) / dt

```

```

33         dudt[k][j][-1] = (u[k, j, - 1] - u[k
    , j, -2]) / dt
34         ##
35         dudx = (u[:, 2:u.size()[1]-1, :] - u[:, 0:u.size
    ()[1]-3, :]) / (2*dx)
36         du2dx2 = (u[:, 2:u.size()[1]-1, :] + u[:, 0:u.
    size()[1]-3, :] - 2 * u[:, 1:u.size()[1]-2, :]) / dx
    ** 2
37         dudt = (u[:, :, 2:u.size()[1]-1] - u[:, :, 0:u.size
    ()[1]-3]) / (2 * dt)'''
38
39         dudx[:, 1:x.shape[1]-2, :] = (u[:, 2:u.size()[1] -
    1, :] - u[:, 0:u.size()[1] - 3, :]) / (2 * dx)
40         du2dx2[:, 1:x.shape[1]-2, :] = (u[:, 2:u.size()[1]
    ] - 1, :] + u[:, 0:u.size()[1] - 3, :] - 2 * u[:, 1:u
    .size()[1] - 2, :]) / dx ** 2
41         dudt[:, :, 1:t.shape[2]-2] = (u[:, :, 2:u.size()[
    1] - 1] - u[:, :, 0:u.size()[1] - 3]) / (2 * dt)
42
43         dudx[:, 0, :] = (u[:, 1, :] - u[:, 0, :]) / (dx)
44         dudx[:, -1, :] = (u[:, -2, :] - u[:, -1, :]) / (
    dx)
45
46         dudt[:, :, 0] = (u[:, :, 1] - u[:, :, 0]) / ( dt)
47         dudt[:, :, -1] = (u[:, :, -2] - u[:, :, -1]) / (
    dt)
48
49         #du2dx2[:, 0, :] = (dudx[:, 1, :] - dudx[:, 0, :])/dx
50         #du2dx2[:, -1, :] = (dudx[:, -2, :] - dudx[:, -1
    , :]) / dx
51
52         #print(dudt.size(), dudx.size(), du2dx2.size())
53
54
55         resid = dudt +0.5* dudx**2 - nu * du2dx2
56         loss = torch.nn.MSELoss()
57         loss = loss(resid, zero_m)
58

```



```
59     bc_loss = torch.nn.MSELoss()
60     bc_loss = bc_loss(u[:,0,:],u[:, -1,:])
61
62     bc_loss2 = torch.nn.MSELoss()
63     bc_loss2 = bc_loss2(dudx[:,0,:],dudx[:, -1,:])
64
65     return loss+bc_loss+bc_loss2
66
67
68 def burgers_data_loss(predicted, target):
69     # Relative L2 Loss
70     # Predicted: (B, Nx, Nt)
71     # Target: (B, Nx, Nt)
72     # TODO
73     num = torch.norm(predicted - target, p =2)
74     den = torch.norm(target,p=2)
75     return num/den
76
77
```

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 import torch
4 import torch.nn.functional as nnF
5 import torch.optim as optim
6
7 from train import *
8 from pde import burgers_data_loss
9 device = torch.device('cuda' if torch.cuda.
    is_available() else 'cpu')
10 batch_size = 3
11 burgers_validation = BurgersDataset(
12     'data/Burgers_test_50_visc_0.01.mat', train=
    False)
13 training_losses = np.load("training-pde_only_losses.
    npy")
14 #model = torch.load('model.pth')
15 model = torch.load('model_pde_only_losses.pth',
    map_location=torch.device('cpu'))
16 model.eval()
17 validation_loader = DataLoader(burgers_validation,
    batch_size=batch_size, shuffle=False)
18 #print(validation_loader)
19 val_loss = 0.0
20 num_val_batches = 0
21
22 with torch.no_grad():
23     for x_val, target_val in validation_loader:
24         x_val, target_val = x_val.to(device),
            target_val.to(device)
25         x_val = x_val.float()
26         val_pred = model(x_val)
27         print(x_val.shape)
28         loss_val = burgers_data_loss(val_pred,
            target_val)
29         val_loss += loss_val.item()
30         num_val_batches += 1
31     break
```

```
32 print(val_pred.shape, target_val.shape)
33 print(num_val_batches)
34
35 fig, (ax1, ax2) = plt.subplots(1,2)
36 countour1 = ax1.contourf(val_pred[0,:,:])
37 countour1_t = ax2.contourf(target_val[0,:,:])
38 ax1.set_title('Predicted Solution PDE only')
39 ax2.set_title('Target Solution')
40 plt.tight_layout()
41 plt.show(block=False)  # Use non-blocking show
42
43 fig2, (ax1, ax2) = plt.subplots(1,2)
44 countour2 = ax1.contourf(val_pred[1,:,:])
45 countour2_t = ax2.contourf(target_val[1,:,:])
46 ax1.set_title('Predicted Solution PDE only')
47 ax2.set_title('Target Solution')
48 plt.tight_layout()
49 plt.show(block=False)
50
51 fig3, (ax1, ax2) = plt.subplots(1,2)
52 countour3 = ax1.contourf(val_pred[2,:,:])
53 countour3_t = ax2.contourf(target_val[2,:,:])
54 ax1.set_title('Predicted Solution PDE only')
55 ax2.set_title('Target Solution')
56 plt.tight_layout()
57 plt.show(block=False)
58
59 training_pde = np.load(f'training-pde_only_losses.npy
    ')
60 plt.figure()
61 plt.plot(np.linspace(0, len(training_pde), len(
    training_pde)-1), training_pde[1:])
62 plt.show()
```

```
1 import torch
2 import torch.nn.functional as nnF
3 import torch.optim as optim
4 from tqdm import tqdm
5 import matplotlib.pyplot as plt
6 import numpy as np
7
8 from data import BurgersDataset
9 from model import ConvNet2D
10 from pde import burgers_pde_residual,
    burgers_data_loss
11 from torch.utils.data import DataLoader
12
13 device = torch.device('cuda' if torch.cuda.
    is_available() else 'cpu')
14
15
16 def pde_training():
17     burgers_train = BurgersDataset(
18         'data/Burgers_train_1000_visc_0.01.mat',
19         train=True)
20     burgers_validation = BurgersDataset(
21         'data/Burgers_test_50_visc_0.01.mat', train=
22         False)
23
24     # Hyperparameters
25     lr = 5e-3
26     batch_size = 16
27     epochs = 40
28
29     # Setup optimizer, model, data loader etc.
30     # TODO
31     train_loader = DataLoader(burgers_train,
32                               batch_size=batch_size, shuffle=True)
33     validation_loader = DataLoader(burgers_validation
34                                   , batch_size=batch_size, shuffle=False)
35
36     alpha = 0
```

```

33     model = ConvNet2D().to(device)
34     optimizer = optim.Adam(model.parameters(), lr=lr)
35     training_losses = []
36     pde_resid_loss = []
37     #loss_pde_only = burgers_pde_residual(inputs
    [::,::,0], inputs[:,::,1], target_val)
38     # Training Loop
39     for epoch in range(epochs):
40         model.train()
41         running_loss = 0.0
42         epoch_loss = 0.0
43         running_loss_pde = 0
44         for batch in train_loader:
45             inputs, targets = batch
46             inputs = inputs.float()
47             loss_pde_only = burgers_pde_residual(
    inputs[:, :, :, 0], inputs[:, :, :, 1], targets)
48             #print(inputs.shape)
49             print(loss_pde_only)
50
51             running_loss_pde += loss_pde_only
52
53             pde_resid_loss.append(running_loss_pde)
54
55             print(f"Epoch {epoch + 1}, Loss: {epoch_loss:
    .4f}")
56         np.save(f'pde_resid_loss.npy', np.array(
    pde_resid_loss))
57         # Validation Loop
58         # TODO
59
60 if __name__ == '__main__':
61     torch.manual_seed(0)
62     pde_training()
63

```

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 import torch
4 import torch.nn.functional as nnF
5 import torch.optim as optim
6
7 from train import *
8 from pde import burgers_data_loss
9 device = torch.device('cuda' if torch.cuda.
    is_available() else 'cpu')
10 batch_size = 3
11 burgers_validation = BurgersDataset(
12     'data/Burgers_test_50_visc_0.01.mat', train=
    False)
13 training_losses = np.load("training-both_losses.npy")
14 #model = torch.load('model.pth')
15 model = torch.load('model_both_losses.pth',
    map_location=torch.device('cpu'))
16 model.eval()
17 validation_loader = DataLoader(burgers_validation,
    batch_size=batch_size, shuffle=False)
18 #print(validation_loader)
19 val_loss = 0.0
20 num_val_batches = 0
21
22 with torch.no_grad():
23     for x_val, target_val in validation_loader:
24         x_val, target_val = x_val.to(device),
            target_val.to(device)
25         x_val = x_val.float()
26         val_pred = model(x_val)
27         print(x_val.shape)
28         loss_val = burgers_data_loss(val_pred,
            target_val)
29         val_loss += loss_val.item()
30         num_val_batches += 1
31         break
32 print(val_pred.shape, target_val.shape)
```

```
33 print(num_val_batches)
34
35 fig, (ax1, ax2) = plt.subplots(1,2)
36 countour1 = ax1.contourf(val_pred[0,:,:])
37 countour1_t = ax2.contourf(target_val[0,:,:])
38 ax1.set_title('Predicted Solution PDE and Supervision
, alpha = 1')
39 ax2.set_title('Target Solution')
40 plt.tight_layout()
41 plt.show(block=False) # Use non-blocking show
42
43 fig2, (ax1, ax2) = plt.subplots(1,2)
44 countour2 = ax1.contourf(val_pred[1,:,:])
45 countour2_t = ax2.contourf(target_val[1,:,:])
46 ax1.set_title('Predicted Solution PDE and Supervision
, alpha = 1')
47 ax2.set_title('Target Solution')
48 plt.tight_layout()
49 plt.show(block=False)
50
51 fig3, (ax1, ax2) = plt.subplots(1,2)
52 countour3 = ax1.contourf(val_pred[2,:,:])
53 countour3_t = ax2.contourf(target_val[2,:,:])
54 ax1.set_title('Predicted Solution PDE and Supervision
, alpha = 1')
55 ax2.set_title('Target Solution')
56 plt.tight_layout()
57 plt.show(block=False)
58
59 training_both = np.load(f'training-both_losses.npy')
60 plt.figure()
61 plt.plot(np.linspace(0,len(training_both),len(
training_both)-1),training_both[1:])
62 plt.show()
63
64 training_rmse_both = np.load(f'training-
rmse_both_losses.npy')
65 print(training_rmse_both)
```